

ComicCraft: AI Comic Story Creator using Gemini Models

Team ID: 6aba3aa9ca93a27509644a39

Project Description:

ComicCraft is a web-based application that uses AI to generate personalized comic book stories and illustrations from user-provided prompts. Built with FastAPI and integrated with Google's Gemini AI models along with Stable Diffusion, ComicCraft streamlines the creative process of generating storylines, dialogues, and vivid comic-style imagery automatically.

The application takes in user information such as the story prompt, main character name, setting, tone, and art style, and generates a panel-by-panel storyline with corresponding illustrations. Users can view the comic preview directly on the web interface and download the full comic as a PDF, structured with both images and narration.

With a clean, interactive HTML interface and efficient image generation powered by Hugging Face's Diffusers library, ComicCraft ensures that each story feels cohesive and visually engaging. It offers both API endpoints and an intuitive frontend, making comic creation accessible to non-artists and casual users alike.

Scenarios

Scenario 1: A user enters a prompt describing their comic story, such as "A brave fox exploring an enchanted forest." The user also specifies preferences such as the main character's name, the setting (e.g., forest), the tone (e.g., dramatic), and the art style (e.g., anime). The system uses **Gemini Flash** to outline the panels, **Gemini Pro** to write the narration and dialogue, and **Stable Diffusion** to create the illustrations, producing a cohesive, personalized comic strip that fits the user's creative vision.

Scenario 2: A user wants a light-hearted, cartoonish feel for their comic. On the input form, they select "funny" as the tone and "comic book" as the art style. When they submit, the system regenerates the entire pipeline (outline, story, and images) with prompts tailored to a more humorous narrative and a classic comic-book look. This lets users iterate on the mood and visual style of their comic until it matches their vision.

Scenario 3: After reviewing the on-screen preview of their comic, the user decides to download it. The system then:

- **Layout Binding:** uses `layout_builder.py` to assemble each panel's title, image, and text into a structured layout.
- **PDF Generation:** calls `exporters.py` to compile the comic into a PDF using FPDF, placing images and narrative text onto consecutive pages and saving the file with a timestamped

filename.

- **Export Success:** the user is redirected to an export success page confirming the download, so the comic can be saved, printed, or shared in a professional PDF format.

Technical Architecture:

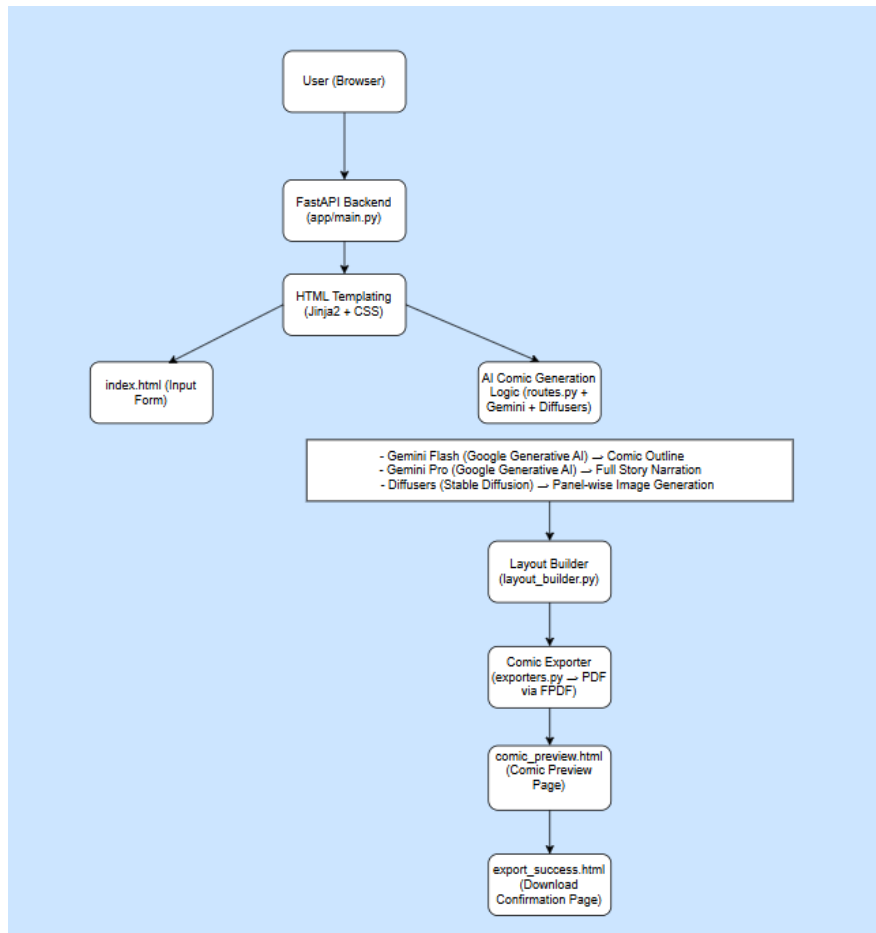


Figure: ComicCraft system architecture and request flow

Description: ComicCraft follows a modular three-tier architecture. The **frontend** (HTML, CSS, and Jinja2 templates) collects the story prompt, character name, setting, tone, and art style, and displays the generated comic. The **FastAPI backend** validates the input, routes requests, and orchestrates the generation pipeline. The **AI integration layer** uses Google Gemini Flash for the panel outline, Gemini Pro for narration and dialogue, and Stable Diffusion (Hugging Face Diffusers) for panel illustrations. The layout builder and PDF exporter then assemble the results into a downloadable comic.

(Note: ComicCraft does not use a database. Generated images and exported PDFs are stored as files in the static/panels and static/exports folders, so no ER diagram is required.)

Pre-requisites:

- **FastAPI Framework Knowledge:** [FastAPI Documentation](#)
- **Uvicorn ASGI Server:** [Uvicorn Docs](#)

- **Python Programming Proficiency:** [Python Official Docs](#)
- **HTML, CSS, and Jinja2 Template Skills:** [W3Schools HTML/CSS/Jinja2 Tutorials](#)
- **Google Gemini API Familiarity:** [Google Generative AI Documentation](#)
- **Hugging Face Diffusers & Stable Diffusion:** [Hugging Face Diffusers Documentation](#)
- **PyTorch Basics:** [PyTorch Documentation](#)
- **PDF Generation with FPDF:** [FPDF Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Python Package & Environment Setup:** [Virtualenv Guide](#)

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Generate the Gemini API key and Hugging Face token and configure them securely in the project environment.
- **Activity 1.2:** Research and select the appropriate generative AI models for story and image generation.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between the frontend, backend, and AI integration.
- **Activity 1.4:** Set up the development environment, installing the necessary libraries and dependencies.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the core functionalities: outline, story, image generation, layout building, and PDF export.
- **Activity 2.2:** Implement the FastAPI backend to manage routing and user input processing.

Milestone 3: routes.py Development

- **Activity 3.1:** Write the main application logic in routes.py, establishing routes for each feature and integrating AI workflows.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the user interface using HTML and CSS.
- **Activity 4.2:** Create dynamic templates with FastAPI's Jinja2 to display results based on user interactions.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment.
- **Activity 5.2:** Test and verify the local deployment.

Milestone 6: Conclusion

Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate generative AI models for the ComicCraft application. This involves researching the capabilities and performance of multiple models to ensure that the chosen models align with the application's objectives of generating comic storylines, character dialogues, scene descriptions, and comic-style illustrations.

Activity 1.1: Generate API Keys

Before the application can communicate with the AI services, you need a Google Gemini API key and a Hugging Face access token. Follow the steps below to create them and connect them securely to the project.

- **Step 1 — Gemini API key:** Sign in to Google AI Studio (aistudio.google.com/app/apikey), click "Create API key", and copy the key immediately.
- **Step 2 — Hugging Face token:** Create a free account at huggingface.co, open Settings — Access Tokens (huggingface.co/settings/tokens), create a new token, and copy it.
- **Step 3 — Add the keys to your project:** Create a .env file in the project root and paste the keys as shown below:

```
GEMINI_API_KEY=your_gemini_api_key_here  
HF_API_KEY=your_huggingface_api_key_here
```

- **Step 4 — Load the keys:** The application loads these variables at startup using python-dotenv.
- **Important — Never commit your API keys:** Add .env to your .gitignore file to prevent accidentally pushing the keys to a public repository:

```
.env
```

Activity 1.2: Research and Select the Appropriate Generative AI Model

Understand the Project Requirements

Review the specific needs of the ComicCraft application, focusing on the types of content it must generate:

- Structured comic outlines (panel by panel)
- Full comic narration and character dialogues
- Visual comic-style illustrations for each panel

The system must deliver creative, engaging, and visually rich comic outputs based on users' custom story prompts.

Model Evaluation

Research the documentation and capabilities of:

- **Google's Gemini models:** Gemini Flash (optimized for fast structured outputs) and Gemini Pro (optimized for detailed, creative text generation).
- **Hugging Face Diffusers:** Stable Diffusion (high-quality AI-generated images from text prompts).

Compare the candidate models based on:

- Response quality for creative writing tasks (narration and dialogues)
- Prompt adherence and richness of detail for scene descriptions
- Speed and resolution of image generation
- Model stability, available documentation, and community support

Practical testing was conducted with different prompts to assess each model's storytelling abilities and image generation performance.

Final Model Selection

- **Gemini Flash** (`models/gemini-1.5-flash`): selected for fast, structured generation of comic panel outlines from story prompts.
- **Gemini Pro** (`models/gemini-1.5-pro`): selected for detailed, creative comic narration and engaging character dialogues.
- **Stable Diffusion** (`runwayml/stable-diffusion-v1-5`): selected for generating high-quality comic-style illustrations from image prompts.

This combination lets ComicCraft deliver rich, creative storytelling alongside vivid, visually appealing comic panels in an automated and efficient manner.

Activity 1.3: Define the Architecture of the Application

The architecture of ComicCraft is structured into three primary components, supported by an image generation stage:

- **Frontend** (HTML, CSS, Jinja2): captures user inputs and displays the generated comics.
- **Backend** (FastAPI): handles data processing and routing.
- **AI Integration** (Google Gemini APIs and Hugging Face Diffusers): generates the story, characters, dialogue, and comic-style images for each scene.

Frontend Responsibilities

Users interact with a simple, user-friendly web interface built with HTML, CSS, and Jinja2 templates. The key input fields are:

- **Story Prompt:** the main idea for the comic
- **Character Name:** the hero of the comic

- **Setting:** the location (e.g., forest, school, city)
- **Story Tone:** the mood of the story (e.g., dramatic, funny)
- **Art Style:** the visual style preference (e.g., anime, realistic)

Upon form submission, the data is sent to the FastAPI backend via a POST request. The templates used are:

- `index.html`: collects user input
- `comic_preview.html`: displays the generated comic
- `export_success.html`: shows a success message after the comic is exported

Backend Responsibilities

FastAPI manages server-side operations and route handling. Its key tasks are to:

- Receive form data from the frontend or JSON API calls.
- Call the AI models to generate panel outlines, story narration, and illustrations.
- Organize and build the complete comic layout.
- Export the comic to a downloadable PDF.

Routes are defined in `routes.py` to handle form-based and JSON-based comic generation, image testing, and comic export confirmation.

AI Integration Points

- **Gemini Flash** generates the structured 5-panel comic outline from the user's story prompt.
- **Gemini Pro** creates detailed narration and character dialogue from the outline.
- **Stable Diffusion** generates comic-style illustrations from the AI-generated image prompt of each panel.
- The backend receives the AI responses, formats the text and images, and passes them to the frontend templates for preview and export.

All AI model interactions happen dynamically during comic generation, so every comic is unique and customized to the user's input.

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure a recent version of Python 3 (3.9 or higher recommended) is installed along with pip, which manages the project dependencies.
- **Create a Virtual Environment:** This keeps project packages from conflicting with system-wide installations:

```
python -m venv comiccraft-env
comiccraft-env\Scripts\activate    # Windows
source comiccraft-env/bin/activate  # macOS / Linux
```

- **Install Required Libraries:**

```
pip install fastapi uvicorn jinja2 python-multipart google-generativeai
pip install diffusers transformers accelerate torch fpdf Pillow python-dotenv
```

- **Run the Server:**

```
uvicorn app.main:app --reload
```

Visit <http://127.0.0.1:8000> for the app and <http://127.0.0.1:8000/docs> for API testing.

- **Set Up the Application Structure:** Create the initial project directory structure for ComicCraft:

```
comiccraft/
|
├─ app/
|   ├── main.py
|   ├── routes.py
|   ├── gemini_flash.py
|   ├── gemini_pro.py
|   ├── image_generator.py
|   ├── layout_builder.py
|   └── exporters.py
|
├─ templates/
|   ├── index.html
|   ├── comic_preview.html
|   └── export_success.html
|
├─ static/
|   ├── panels/      # Generated panel images
|   ├── exports/     # Exported comic PDFs
|   └── fonts/       # Fonts for PDF generation
|
├─ .env              # Environment variables (API keys)
└─ requirements.txt  # Python dependencies list
```

Figure: ComicCraft project structure

Milestone 2: Core Functionalities Development

Activity 2.1: Develop the Core Functionalities

Generate a Structured Comic Panel Outline

Function: generate_outline() **File:** gemini_flash.py

- Uses the **Gemini Flash** model to generate a structured 5-panel comic outline from the user's story prompt.
- Each panel contains a panel number, title, scene description, and an image generation prompt.
- The output is a list of dictionaries, one for each comic panel.

```
def generate_outline(user_prompt: str) -> list:
    """
    Generates a 5-panel comic layout based on the user's story idea using Gemini.

    Args:
        user_prompt (str): The user's comic idea prompt.

    Returns:
        list: A list of dictionaries, one for each panel.
    """
    prompt = f"""
You are a professional AI comic planner.

Your task is to generate a *strictly formatted* JSON array containing 5 panel descriptions for a comic based on the story idea below:

STORY: "{user_prompt}"

Each JSON object must include:
- "panel" (integer)
- "title" (string)
- "scene_description" (string)
- "image_prompt" (string)

Respond ONLY in this valid JSON format, without any explanations or markdown:
[
    {{
        "panel": 1,
        "title": "Title here",
        "scene_description": "Scene description here",
        "image_prompt": "Image prompt for Stable Diffusion"
    }},
    ...
]
    """
```

```
try:
    response = model.generate_content(prompt)
    output_text = response.text.strip()

    print("\n 🟡 RAW GEMINI RESPONSE 🟡 \n", output_text)

    # Remove any markdown formatting if present
    if output_text.startswith("```json"):
        output_text = output_text.replace("```json", "").replace("```", "").strip()

    panel_data = json.loads(output_text)

    # Additional structure validation
    if not isinstance(panel_data, list):
        raise ValueError("Gemini response is not a list.")

    for panel in panel_data:
        if not isinstance(panel, dict) or not all(key in panel for key in ("panel", "title", "scene_description", "image_prompt")):
            raise ValueError(f"Invalid panel format or missing keys: {panel}")

    return panel_data

except json.JSONDecodeError as e:
    print("❌ JSON Decode Error:", e)
    print("❌ Full Text Received:\n", output_text)
    return [{"error": f"JSON parsing failed: {str(e)}"}]

except Exception as e:
    print("❌ Unexpected Error:", e)
    return [{"error": f"Generation failed: {str(e)}"}]
```

Code: generate_outline() in gemini_flash.py

Generate Comic Story Narration and Dialogue

Function: generate_story() **File:** gemini_pro.py

- Uses the **Gemini Pro** model to expand the panel outlines into a full comic-style story.
- Creates engaging narration and character dialogue for each panel.
- The output is a single formatted text containing the stories of all panels.

```
def generate_story(outline: list) -> str:
    """
    Generates a detailed comic story with narration and character dialogue
    from a list of comic panel outlines using Gemini 1.5 Pro.

    Args:
        outline (list): A list of strings representing each comic panel's idea.

    Returns:
        str: The generated comic story text or an error message.
    """

    # Format the panel outline as a numbered list for clarity
    formatted_outline = "\n".join([f"{i+1}. {item}" for i, item in enumerate(outline)])

    # Construct the prompt
    prompt = f"""
    You're a comic book writer.

    Given the following panel breakdown, write a comic-style story with engaging narration and character dialogues for each panel.

    Panel Outline:
    {formatted_outline}

    Guidelines:
    - Use a fun and engaging tone, like an actual comic book.
    - Include narration and clearly marked character lines.
    - Keep each panel self-contained but part of a cohesive story.
    """

    try:
        response = model.generate_content(prompt)
        return response.text
    except Exception as e:
        return f"Error generating story: {str(e)}"
```

Code: generate_story() in gemini_pro.py

Generate Comic Illustrations from Prompts

Function: generate_image() **File:** image_generator.py

- Uses the **Stable Diffusion** model to create a comic-style image from the provided image prompt.
- Sanitizes the prompt into a safe filename and saves the generated image to the static/panels directory.
- Returns the file path of the saved image.

```
def generate_image(prompt, filename=None):
    if not filename:
        filename = sanitize_filename(prompt)

    image = pipe(prompt).images[0]
    path = f"static/panels/{filename}"
    os.makedirs(os.path.dirname(path), exist_ok=True)
    image.save(path)
    return path
```

Code: generate_image() in image_generator.py

Organize Comic Panels into a Layout Structure

Function: build_comic_layout() **File:** layout_builder.py

- Organizes the generated images and the full comic story into a structured layout.
- Matches each image with its corresponding panel story.
- The output is a list of dictionaries containing the panel number, image path, and text.

```
def build_comic_layout(image_paths, full_story, outline):
    # Split the full story into individual panel segments
    story_panels = full_story.split("**Panel")
    story_panels = [f"**Panel{panel}" for panel in story_panels if panel.strip()]

    # Build layout panel by panel
    layout = []
    for idx, (image, text, panel_info) in enumerate(zip(image_paths, story_panels, outline), start=1):
        layout.append({
            "panel": idx,
            "title": panel_info.get("title", f"Panel {idx}"),
            "image_path": image,
            "text": "\n".join(text.strip().splitlines()[1:]).strip(), # remove panel title
            "scene_description": panel_info.get("scene_description", "")
        })

    return layout
```

Code: build_comic_layout() in layout_builder.py

Export the Comic Panels and Story to PDF

Function: save_pdf() **File:** exporters.py

- Compiles the full comic into a multi-page PDF using the FPDF library.
- Places each panel's image and narration neatly on separate pages.
- Saves the final PDF in the static/exports folder and returns its path.

```
def save_pdf(layout):
    pdf = PDF()
    pdf.set_auto_page_break(auto=True, margin=15)

    # Add Unicode font
    pdf.add_font('Dejavu', '', FONT_PATH, uni=True)
    pdf.set_font('Dejavu', '', 12)

    for panel in layout:
        image_path = panel['image_path']
        story_text = panel['text']

        pdf.add_page()

        # Panel title (no bold)
        pdf.set_font('Dejavu', '', 14)
        pdf.cell(0, 10, f"Panel {panel['panel']}", ln=True, align="C")
        pdf.set_font('Dejavu', '', 12)

        # Image placement
        y_image = 30
        image_height = 100
        spacing_after_image = 15

        if os.path.exists(image_path):
            pdf.image(image_path, x=10, y=y_image, w=pdf.w - 20, h=image_height)
        else:
            pdf.set_y(y_image)
            pdf.multi_cell(0, 10, f"Image missing: {image_path}")

        # Text placement below image
        pdf.set_y(y_image + image_height + spacing_after_image)
        story_lines = story_text.strip().splitlines()
        # Remove title line like "***Panel 1: Title***" if present
        if story_lines and story_lines[0].strip().lower().startswith("***panel"):
            story_lines = story_lines[1:]
        cleaned_text = "\n".join(story_lines).strip()
        pdf.multi_cell(0, 10, cleaned_text)

    # Save with timestamp
    timestamp = datetime.now().strftime("%Y%m%d%H%M%S")
    filename = f"comic_{timestamp}.pdf"
    pdf_path = os.path.join(EXPORT_FOLDER, filename)
    pdf.output(pdf_path)

    return pdf_path
```

Code: save_pdf() in exporters.py

Activity 2.2: Implement the FastAPI Backend

Define Routes in FastAPI

All routing logic is handled in `routes.py`. Each route is linked to the backend function that runs the comic generation workflow developed in Activity 2.1.

Process User Input

- **HTML Form:** created in index.html.
- **Users enter:** story prompt, main character name, setting, tone, and art style.
- **FastAPI captures inputs** using Form(...) parameters from the request.
- **Pydantic schema** (PromptRequest) validates JSON input for API-based comic generation.

Integrate Gemini API Calls

Each route handler connects directly to the respective AI-powered function modules:

- **/generate**
 - Calls generate_outline() to create panel outlines from the user prompt.
 - Calls generate_story() to expand them into narration and dialogue.
 - Calls generate_image() for each panel to create comic-style images.
 - Calls build_comic_layout() to organize the final layout.
 - Calls save_pdf() to compile and export the complete comic as a PDF.
- **/generate-comic/json**
 - Follows the same flow as /generate, but is triggered by a raw JSON API request.
 - Returns a JSON response with the layout data and PDF path for API clients.
- **/test-image**
 - Calls generate_image() directly with a custom prompt to test image generation separately.

All AI-generated responses (text and images) are processed, cleaned, and injected into the Jinja2 templates (comic_preview.html and export_success.html) for a seamless user experience.

Milestone 3: routes.py Development

In this milestone, the primary focus is on setting up all major FastAPI routes for the app, handling user inputs efficiently, integrating the AI generation workflows, and ensuring smooth interaction between backend processing and frontend templates. This step-by-step approach creates a stable foundation for processing prompts, generating comics, and providing a seamless user experience.

Activity 3.1: Writing the Main Application Logic in routes.py

Define the Core Routes in routes.py

Separate routes are established for each of ComicCraft's core functionalities:

- `/` — loads the homepage (`index.html`) where users submit their story details.
- `/generate` — handles form submission, processes the input using AI, generates the comic panels, and returns the comic preview page.
- `/generate-comic/json` — an API route that accepts JSON payloads, triggers comic generation, and returns the comic layout data and the generated PDF path.
- `/export-success` — displays a success confirmation page after the comic is downloaded.
- `/test-image` — a developer utility route that tests image generation from a direct prompt without full comic creation.

Each route serves as a distinct entry point for a different type of interaction, through browser forms or direct API requests.

Set Up Route Handlers for Each Feature

- For `/generate`, user inputs are captured using `Form(...)` parameters: story prompt, character name, setting, story tone, and art style.
- For `/generate-comic/json`, inputs are captured from a JSON body using the `PromptRequest` Pydantic model.
- Form data and JSON payloads are passed to the backend functions (`generate_outline`, `generate_story`, `generate_image`, `build_comic_layout`, `save_pdf`) for AI processing.
- Error handling uses FastAPI's `HTTPException` and `try-except` blocks to gracefully handle unexpected failures such as API errors or JSON decode errors.

Integrate AI Workflows in Each Route

In `/generate` and `/generate-comic/json`, the core AI functionalities are integrated as follows:

- `generate_outline()` is called to create a 5-panel structured story.
- `generate_story()` is called to write the narration and dialogues.
- `generate_image()` is called iteratively to generate an image for each panel.
- `build_comic_layout()` structures the final panel layout.

- `save_pdf()` compiles the story and illustrations into an exportable PDF.

Each AI-generated output is passed dynamically to the HTML template (`comic_preview.html`) or returned as a JSON response for API clients. Users receive a complete, coherent comic experience, while developers and testers can interact with the backend directly through structured API responses.

Comic Generation Route:

```

21
22
23 @router.post("/generate", response_class=HTMLResponse)
24 async def generate_comic(
25     request: Request,
26     prompt: str = Form(...),
27     character_name: str = Form(...),
28     setting: str = Form(...),
29     tone: str = Form(...),
30     style: str = Form(...)
31 ):
32     try:
33         # Combine user input into a single full prompt
34         full_prompt = (
35             f"{prompt}\n"
36             f"The main character is {character_name}. "
37             f"The setting is a {setting}. "
38             f"The tone is {tone}. The art style is {style}."
39         )
40
41         # Step 1: Generate panel outline
42         outline = generate_outline(full_prompt)
43
44         if not isinstance(outline, list) or not all("image_prompt" in panel for panel in outline):
45             raise ValueError("Invalid outline structure from Gemini response.")
46
47         # Step 2: Generate story
48         full_story = generate_story(outline)
49
50         # Step 3: Generate images
51         images = [generate_image(panel["image_prompt"]) for panel in outline]
52
53         # Step 4: Build layout
54         layout = build_comic_layout(images, full_story, outline)
55
56         # Step 5: Export to PDF
57         pdf_path = save_pdf(layout)
58         web_pdf_path = "/" + pdf_path.replace("\\", "/") # Make path safe for web
59
60         return templates.TemplateResponse("comic_preview.html", {
61             "request": request,
62             "layout": layout,
63             "pdf_path": web_pdf_path
64         })
65
66     except Exception as e:
67         traceback.print_exc()
68         raise HTTPException(status_code=500, detail=str(e))
69

```

Code: /generate route in routes.py

Export Success Route:

```

70
71 @router.get("/export-success", response_class=HTMLResponse)
72 async def export_success(request: Request, pdf_path: str):
73     return templates.TemplateResponse("export_success.html", {
74         "request": request,
75         "pdf_path": pdf_path
76     })
77
78

```

Code: /export-success route in routes.py

Image Generation Route:

```
120
121 @router.get("/test-image")
122 async def test_image(prompt: str = "A futuristic city at sunset, sci-fi, cinematic, artstation"):
123     try:
124         image_path = generate_image(prompt)
125         return {"message": "Image generated successfully", "path": image_path}
126     except Exception as e:
127         raise HTTPException(status_code=500, detail=str(e))
128
```

Code: /test-image route in routes.py

Milestone 4: Frontend Development

This milestone involves building responsive, clean web pages using HTML, CSS, and Jinja2 templating. Each template is dynamically rendered through FastAPI, so users can easily submit inputs, view generated comics panel by panel, and confirm comic export actions. This guarantees an engaging and seamless comic creation experience.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure

`index.html` is developed as the main entry point for user input. The form captures the story prompt, main character name, setting, story tone, and art style. Each field is properly labeled and grouped using semantic HTML elements. The design uses a scenic background image, simple typography, and an organized form layout to encourage creativity, with minimal navigation so the flow goes directly from input to comic generation.

Design a Responsive Layout Using CSS

- CSS styles are embedded directly within each HTML file.
- Layouts use a fixed width with centered content to keep the pages readable across screen sizes.
- Buttons share a consistent style with hover effects.
- Input fields and result panels use padding, box shadows, and rounded corners for a clean, modern look.
- A light color scheme enhances visibility and gives the interface an inviting feel.

Create Separate Pages for Each Core Functionality

Three Jinja2-powered HTML templates are created under the `/templates` folder:

- **`index.html`**: a form where users enter the story prompt, character name, setting, tone, and preferred art style. It submits the data to the `/generate` route via POST.
- **`comic_preview.html`**: displays the AI-generated comic panel by panel, so users can review the full storyline sequentially. Each panel includes the title, scene illustration, scene description (in italics), and the captions and narration text.
- **`export_success.html`**: displays a success message after the user downloads the comic as a PDF and encourages them to return and create a new comic.

The templates are modular and cleanly separated by functionality, so the interface moves smoothly between comic creation and comic review.

Activity 4.2: Creating Dynamic Templates with FastAPI's Jinja2

Integrate Jinja2 Templating for Dynamic Content Rendering

FastAPI's Jinja2Templates is used for template rendering:

```
from fastapi.templating import Jinja2Templates

templates = Jinja2Templates(directory="templates")
```

Each route uses TemplateResponse to render HTML dynamically based on the user's interaction:

- The / route renders index.html.
- The **/generate** route renders comic_preview.html.
- The **/export-success** route renders export_success.html.

This dynamic binding ensures that the comic content (layout, images, and text) updates automatically after user input.

Bind Backend Data to HTML Templates

- **index.html**
 - Form elements use name="..." attributes that map directly to the backend Form(...) parameters in FastAPI.
 - Submits the form data via POST to /generate.
- **comic_preview.html**
 - Dynamically displays the panel number, title, generated image, scene description (in italics), and the captions and narration text.
 - Loops through the layout list using the Jinja2 {% for panel in layout %} syntax.
- **export_success.html**
 - Displays a confirmation message using Jinja2 variables.
 - Receives the exported PDF path as context and provides a button for returning to the homepage.

The frontend of ComicCraft is fully integrated with FastAPI, dynamically renders AI-generated comics, and provides a smooth user flow from prompt input to comic export.

Milestone 5: Deployment

In Milestone 5, the focus is on deploying the ComicCraft application on a local system using FastAPI. This involves configuring the server environment, installing all required dependencies, setting up static files and templates, and launching the app locally to simulate a real-world environment. It verifies that the complete comic generation workflow (AI text generation, image creation, layout building, and PDF export) runs smoothly before planning a cloud deployment in the future.

Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment

Create a virtual environment to manage dependencies and keep the deployment isolated from other Python projects. Activate it and install all libraries listed in requirements.txt, including FastAPI, Uvicorn, Google Generative AI, Hugging Face Diffusers, FPDF, and others.

```
python -m venv env

# For Windows
env\Scripts\activate

# For macOS/Linux
source env/bin/activate

pip install -r requirements.txt
```

Configure Environment Variables

- Use environment variables to store sensitive data such as the Gemini API key and the Hugging Face API token.
- These variables can be set in the terminal or added to a .env file, which the application loads using python-dotenv:

```
GEMINI_API_KEY=your_gemini_api_key_here
HF_API_KEY=your_huggingface_api_key_here
```

Frontend-Backend Rendering via Jinja2

All user interactions are handled through Jinja2-rendered templates:

- **index.html:** collects the user inputs.
- **comic_preview.html:** displays the generated comic panels.
- **export_success.html:** confirms the successful export of the comic.

Activity 5.2: Testing and Verifying Local Deployment

Start the FastAPI Server

Navigate to the project root where app/main.py is located and launch the FastAPI development server with hot reload enabled:

```
uvicorn app.main:app --reload
```

Access the Application Locally

Once the server is running, open a web browser and navigate to <http://127.0.0.1:8000>. This loads the ComicCraft homepage, where users enter their story prompt, character name, setting, tone, and art style to generate a personalized AI-powered comic.

The interactive API documentation provided by FastAPI is available at <http://127.0.0.1:8000/docs>. It lets developers test the available endpoints: /generate-comic/json, /generate, /export-success, and /test-image.

Exploring the Website's Web Pages:

Home Page:

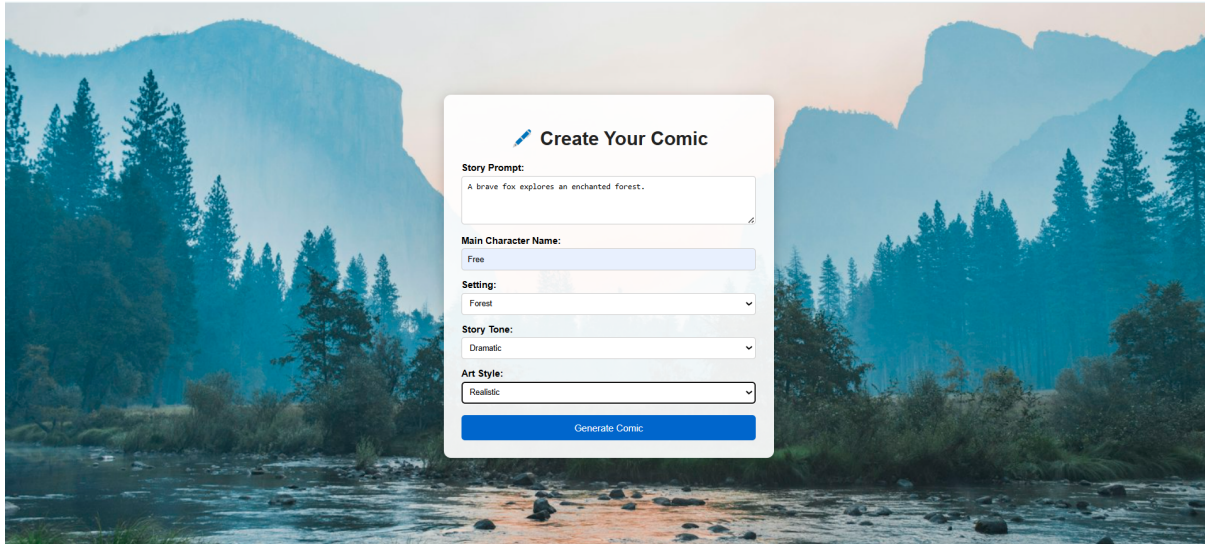


Figure: ComicCraft home page

Description: The homepage of ComicCraft is the starting point for creating AI-generated comic stories. It features a clean layout with a scenic background that fosters a creative atmosphere. At the center, a structured form collects the inputs needed for comic generation. Users can:

- Enter a **Story Prompt** describing their comic idea,
- Specify a **Main Character Name**,
- Choose a **Setting** (school, forest, space, city),
- Select a **Story Tone** (light-hearted, dramatic, poetic, funny),
- Pick an **Art Style** (anime, pixel art, comic book, realistic).

Once the form is submitted, the system uses AI models to generate a complete comic, including the storyline outline, detailed dialogues, illustrations, and an exportable PDF. The page is simple, user-friendly, and visually appealing, making comic creation accessible without technical expertise.

Comic Preview — Result Page:

 Your Comic Preview

Panel 1: The Forest's Edge



Free, a red fox with intelligent eyes, stands at the edge of a dark, imposing forest. Sunlight barely penetrates the dense canopy. He looks apprehensive but determined.

Figure: Comic preview, Panel 1

Panel 2: Into the Deep Woods



Free cautiously enters the forest, the path barely visible beneath twisted branches and overgrown foliage. An unnatural silence hangs in the air.

****CAPTION:**** Whispers carried on the wind, tales of magic and mystery. Tales of the Whispering Woods...

****NARRATION:**** Free, the bravest fox this side of Silverstream, stood at the threshold. The ancient trees loomed like watchful giants, their shadows a daunting promise.

****IMAGE PROMPT:**** Realistic painting of a red fox, Free, standing at the edge of a dark, mysterious forest. Dramatic lighting, sunlight barely penetrating the trees. Focus on the fox's determined expression.

Figure: Comic preview, Panel 2

Description: The Comic Preview page presents the AI-generated comic panels sequentially after the user submits the homepage form, combining visual and textual elements for each panel. Every panel includes:

- **Panel Title:** labeled with the panel number and a descriptive title (e.g., “Panel 2: Into the Deep Woods”).

- **Comic Image:** an AI-generated illustration created from the scene's image prompt.
- **Scene Description:** a short paragraph, displayed in italics, that sets the atmosphere and context.
- **Caption and Narration:** a caption giving a brief, ambient description of the environment or background sounds, and narration detailing the main character's actions, emotions, or dialogue.
- **Image Prompt Reference:** the original artistic description used to generate the illustration, displayed for reference.

Each panel is clearly separated and visually appealing, making it easy to follow the story panel by panel. At this stage the page focuses purely on viewing and reviewing the story before any final action such as exporting.

Download Button:

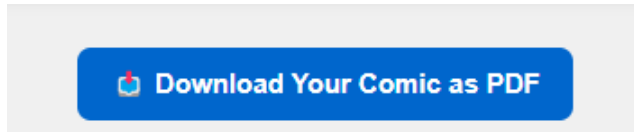


Figure: Download button on the Comic Preview page

Description: On the Comic Preview page, a “Download Your Comic as PDF” button is displayed. When users click it, the entire comic (images, panel titles, scene descriptions, captions, and narration) is compiled into a professionally formatted PDF and downloaded to the user’s device, so it can be saved, shared, and viewed offline.

Comic Export Success Page:

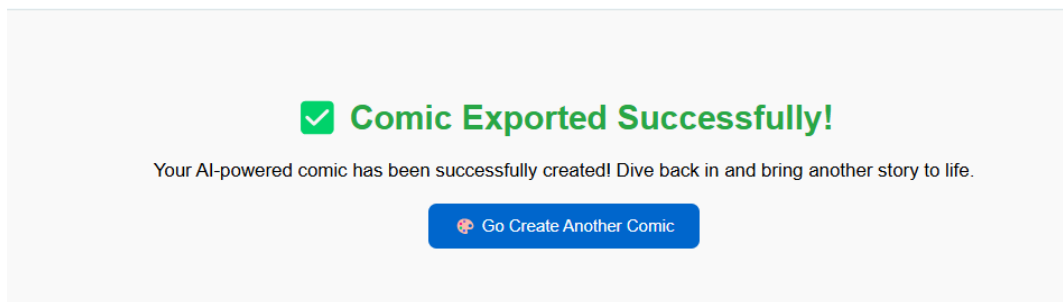


Figure: Comic export success page

Description: After the download is triggered, users are redirected to the Comic Export Success page, which confirms that their AI-powered comic has been created and downloaded. The page includes a “Go Create Another Comic” button that returns users to the homepage to start a new comic. This export-and-success flow gives users closure and a tangible output from their story idea while encouraging continued engagement.

Milestone 6: Conclusion

ComicCraft successfully demonstrates the creative potential of AI-driven comic generation. By combining Large Language Models (Gemini Flash and Gemini Pro) for storytelling with Stable Diffusion for comic-style illustration, the application turns simple prompts into complete, visually engaging comic narratives.

The project provides an end-to-end experience, from personalized story outline generation, full narration, and high-quality image creation to exporting the final comic as a downloadable PDF, all within an intuitive web interface. ComicCraft simplifies comic creation for non-artists, storytellers, and hobbyists, offering a seamless blend of imagination and AI assistance.

While the current version focuses on single comic generation without user account management, its modular architecture allows easy future enhancements such as user profiles, comic libraries, and multi-page story arcs. Overall, ComicCraft shows how AI can democratize storytelling and empower users to bring their creative visions to life with minimal effort.